



河北建筑工程学院

HeBei University Of Architecture

学士学位毕业设计

华锐 SL1500 型双馈式风机齿轮箱
智能健康管理体系

姓 名 刘淑杨

学 号 20223090239

院 系 信息工程学院

专 业 计算机科学与技术

班 级 计 224

指导教师 董颢霞

2026 年 05 月

毕业设计原创性声明

本人所提交的毕业设计《华锐 SL1500 型双馈式风机齿轮箱智能健康管理体系》，是在导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本毕业设计不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。

本声明的法律后果由本人承担。

毕业设计作者（签名）：

年 月 日

指导教师确认（签名）：

年 月 日

毕业设计版权使用授权书

本毕业设计作者完全了解河北建筑工程学院有权保留并向国家有关部门或机构送交毕业设计的复印件和磁盘，允许毕业设计被查阅和借阅。本人授权河北建筑工程学院可以将毕业设计的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编毕业设计。

保密的毕业设计在_____年解密后适用本授权书。

毕业设计作者（签名）：

年 月 日

指导教师（签名）：

年 月 日

摘 要

随着风电装机规模持续扩大，风电场对机组可靠性、可利用率和智能化运维能力提出了更高要求。齿轮箱作为双馈式风力发电机组传动链中的关键部件，长期承受交变载荷、冲击载荷、温度波动和复杂环境影响，容易出现齿面磨损、点蚀剥落、轴承损伤、油温过高、润滑状态劣化、冷却系统异常等故障。一旦齿轮箱故障未能及时发现，不仅会造成限功率运行和非计划停机，还可能引发大部件更换，显著增加吊装、备件和发电损失成本。因此，面向风机齿轮箱建立实时监测、智能诊断和健康管理系统具有重要工程意义。

本文以华锐 SL1500 型双馈式风机齿轮箱为研究对象，结合风机实时数据、故障统计数据和系统演示需求，设计并实现了一套齿轮箱智能健康管理系统。系统采用 Flask 作为后端框架，使用 SQLite 进行数据存储，前端采用 HTML、CSS、JavaScript 实现，并结合 ECharts、Three.js、Server-Sent Events 等技术完成实时状态展示、故障趋势分析和齿轮箱数字孪生示意。系统后端包括数据采集与融合、风机数据仓库、振动特征提取、故障诊断、健康评分、剩余寿命估计、故障记录管理、健康报告生成和运维问答等模块。

在诊断方法方面，系统提取振动 RMS、峰值、峭度、峰值因子、脉冲因子和包络峰值等特征，并融合齿轮箱油温、振动烈度、油液 NAS 等级和历史故障信息计算风险因子。系统根据风险因子和规则库识别齿轮齿面磨损、齿面点蚀/剥落、齿轮断齿、轴承磨损、轴承点蚀/剥落、高速轴承过温、齿轮箱油温过高、润滑油污染/油液劣化、轴系不对中、齿轮啮合异常、箱体或基础松动、冷却系统故障等典型故障，并输出健康评分、RUL、诊断依据和维护建议。系统还构建了本地专家知识库，并预留大模型接口，用于回答油温、振动、轴承、齿轮、润滑、检修周期和 M-IALO-SVR 算法相关问题。

测试结果表明，本文设计的系统能够完成风场总览、单机详情、实时监测、故障诊断、故障代码库、故障数据管理、健康报告、参数设置、用户管理和运维辅助问答等功能，能够为风电场运维人员提供直观的状态感知、风险判断和处置建议。系统目前仍属于毕业设计原型，后续可进一步接入真实 SCADA、CMS 和油液监测系统，并基于真实故障样本训练和验证机器学习模型。

关键词：华锐 SL1500；双馈式风机；齿轮箱；故障诊断；健康管理；RUL；智能运维

ABSTRACT

With the continuous expansion of wind power capacity, wind farms require higher reliability, availability and intelligent operation capability. As a key component in the transmission chain of a doubly-fed wind turbine, the gearbox is exposed to alternating loads, impact loads, temperature fluctuations and complex environmental conditions. It is prone to gear wear, pitting, bearing damage, abnormal oil temperature, lubricant deterioration and cooling system faults. If gearbox faults are not detected in time, they may lead to power limitation, unplanned shutdown and expensive component replacement.

Taking the gearbox of the Sinovel SL1500 doubly-fed wind turbine as the research object, this thesis designs and implements an intelligent health management system. The system adopts Flask as the backend framework, SQLite as the database, and HTML/CSS/JavaScript as the frontend technology stack. ECharts, Three.js and Server-Sent Events are used for real-time monitoring, fault trend analysis and digital twin visualization. The backend integrates data acquisition, wind turbine data repository, vibration feature extraction, fault diagnosis, health score calculation, remaining useful life estimation, fault record management, report generation and maintenance question answering.

The system extracts vibration features such as RMS, peak value, kurtosis, crest factor, impulse factor and envelope peak. It combines gearbox oil temperature, vibration intensity, oil quality and historical fault information to calculate risk factors. Based on a rule-based diagnostic library, the system identifies typical gearbox faults and outputs health score, RUL, diagnostic basis and maintenance suggestions. A local expert knowledge base and an optional large language model interface are also designed for intelligent operation and maintenance assistance.

The test results show that the system can realize wind farm overview, turbine detail monitoring, real-time data display, fault diagnosis, fault code management, fault data management, health report generation, parameter setting, user management and intelligent maintenance Q&A. It provides intuitive status perception and decision support for wind farm operators. The current system is still a graduation design prototype, and future work will focus on real SCADA/CMS/oil monitoring data access and machine learning model validation based on real fault samples.

Key words: SL1500; doubly-fed wind turbine; gearbox; fault diagnosis; health management; RUL; intelligent operation and maintenance

目 录

- 摘要..... I
- ABSTRACT..... II
- 第 1 章 绪论..... 1
 - 1.1 研究背景与意义..... 1
 - 1.2 国内外研究现状..... 1
 - 1.3 本文研究内容..... 2
 - 1.4 技术路线..... 2
- 第 2 章 系统需求分析..... 3
 - 2.1 业务需求分析..... 3
 - 2.2 功能需求分析..... 3
 - 2.3 非功能需求分析..... 4
 - 2.4 数据需求分析..... 4
 - 2.5 需求约束与边界分析..... 4
 - 2.6 关键数据字段设计..... 5
- 第 3 章 相关技术与理论基础..... 7
 - 3.1 齿轮箱典型故障机理..... 7
 - 3.2 多源状态监测技术..... 7
 - 3.3 振动特征提取方法..... 7
 - 3.4 风险评分与健康评估方法..... 7
 - 3.5 Web 系统开发技术..... 8
 - 3.6 健康评分与 RUL 估计原理..... 8
 - 3.7 智能问答与知识库技术..... 9
- 第 4 章 系统总体设计..... 10
 - 4.1 系统架构设计..... 10
 - 4.2 数据库设计..... 10
 - 4.3 后端接口设计..... 11
 - 4.4 诊断流程设计..... 12
 - 4.5 前端页面设计..... 13
- 第 5 章 系统详细实现..... 14
 - 5.1 数据采集与融合模块实现..... 14
 - 5.2 故障诊断算法模块实现..... 15
 - 5.3 风场总览与单机详情实现..... 17
 - 5.4 故障代码库与数据管理实现..... 18
 - 5.5 AI 运维问答模块实现..... 19
 - 5.6 健康报告与工单闭环实现..... 20
- 第 6 章 系统测试与结果分析..... 24
 - 6.1 测试环境..... 24
 - 6.2 功能测试..... 24
 - 6.3 诊断场景测试分析..... 24
 - 6.4 结果评价..... 25

6.5 测试用例设计.....	25
6.6 典型诊断场景结果.....	26
6.7 系统运行效果分析.....	26
6.8 单机详情与 HMI 验证.....	27
6.9 非功能测试与可用性分析.....	27
6.10 存在问题与改进方向.....	28
第 7 章 总结与展望.....	29
参考文献.....	30
致谢.....	31
附录 A 系统主要接口.....	32
附录 B 核心代码片段与接口说明.....	33
附录 C 系统部署与运行说明.....	35

本文基于实际项目代码和风机数据文件，完成华锐 SL1500 型双馈式风机齿轮箱智能健康管理体系的设计与实现。主要研究内容如下：

第一，分析齿轮箱典型故障机理和系统业务需求。结合齿轮箱结构特点，梳理齿轮、轴承、润滑、冷却、轴系和基础结构相关故障类型，明确系统需要支持的状态监测、故障诊断、故障码管理、健康评估和运维问答功能。

第二，设计多源数据接入与融合方法。系统一方面通过 `DataAcquisitionSystem` 模拟 SCADA、CMS 和油液测点，另一方面通过 `WindDataRepository` 读取风机数据摘要文件，融合自由报表、实时数据、故障统计和故障信息统计，形成风场总览、单机详情和趋势分析数据。

第三，设计并实现齿轮箱故障诊断流程。系统对振动信号提取 RMS、峰值、峭度、峰值因子、脉冲因子和包络峰值，结合油温、振动 RMS 和油液 NAS 等指标计算风险因子，再根据故障规则库输出故障类型、严重程度、置信度、健康评分、RUL、诊断依据和维护建议。

第四，设计并实现 Web 健康管理平台。后端使用 Flask 构建接口，前端使用 HTML、CSS、JavaScript、ECharts 和 Three.js 展示系统功能。系统实现了用户登录注册、风场总览、单机详情、实时监测、故障诊断、故障数据管理、健康报告、参数设置、故障代码库、HMI 页面和 AI 运维问答等模块。

第五，对系统功能进行测试与分析。通过不同故障场景、不同机组切换和不同接口调用，验证系统能否稳定展示运行状态、生成诊断结果、保存故障记录、导出数据和输出报告。

1.4 技术路线

本文技术路线可以概括为：需求分析与故障机理研究、数据接入与预处理、特征提取与风险评分、故障诊断与健康评估、后端接口与数据库设计、前端可视化实现、系统测试与结果分析。系统首先获取齿轮箱油温、振动、油液、功率、风速、转速、故障码等数据；其次进行滤波、异常值处理和字段归一化；然后提取振动特征并计算风险因子；最后输出故障诊断、健康评分、RUL、复检周期和工单建议，并通过前端页面展示给运维人员。

第 2 章 系统需求分析

2.1 业务需求分析

风电场运维人员面对的是多台机组、多个数据源和多种故障类型。如果没有统一平台，油温、振动、故障码、检修记录和运行报表往往分散在不同系统或文件中，运维人员需要手动比对，效率较低。本文系统需要实现对多台 SL1500 风机齿轮箱状态的集中管理，支持从风场级总览下钻到单机详情，再进一步进入故障诊断和运维建议。

从日常运维流程看，系统应满足以下业务需求：查看风场所有机组运行状态；识别告警、故障、限功率、待风和维护停机状态；查看单机油温、振动、功率、风速、健康评分和 RUL；根据故障码和指标阈值判断风险等级；运行诊断模型生成维护建议；保存和导出故障记录；生成健康评估报告；通过问答方式获取排查步骤；模拟检修工单闭环。

2.2 功能需求分析

表 2.1 系统功能需求汇总

功能模块	主要功能	项目代码对应
用户与权限	登录、注册、角色显示、管理员默认账号初始化	auth_route.py、database.py
健康监测	风场总览、单机详情、实时状态、振动波形、SSE 推送	windfarm_route.py、data_route.py
智能诊断	特征提取、风险评分、故障分类、健康评分和 RUL 估计	ml_models.py
数据管理	故障记录列表、历史故障统计、CSV 导出	data_route.py、wind_data_repository.py
运维问答	本地知识库检索、实时工况分析、可选大模型增强	rag_service.py、qa_route.py
报告与工单	健康报告、复检建议、P1/P2 工单模拟	report_route.py、ops_route.py

系统功能需求包括用户管理、健康监测、智能诊断、数据管理、报告生成、参数配置和运维问答七个方面。

用户管理模块包括用户登录、注册和角色显示。后端通过 auth_route.py 提供登录注册接口，数据库中的 User 表保存用户名、密码哈希和角色。

健康监测模块包括风场总览、单机详情和实时监测。风场总览展示时间可利用率、能量可利用率、平均油温、平均风速、总功率、平均健康评分、告警数量和运行机组数。单机详情展示有功功率、风速、发电机转速、变桨角、温度、寿命、部件状态、集群对标和告警记录。实时监测页面通过 SSE 接收状态和振动波形。

智能诊断模块包括故障诊断、故障代码库和 AI 运维问答。故障诊断支持正常、齿轮磨损、点蚀剥落、断齿、轴承磨损、轴承点蚀、高速轴承过温、齿轮箱油温过高、油液污染、轴系不对中、齿轮啮合异常、基础松动和冷却故障等场景。故障代码库包括 GBX-001 至 GBX-015 等齿轮箱关联故障，并能结合历史故障统计动态补充现场故障码。

数据管理模块展示诊断记录和历史故障，支持 CSV 导出。报告模块根据当前状态和历史趋势生成健康评分、关键指标、诊断结论、维护计划和工单建议。参数配置模块展示监测采集、齿轮箱阈值、寿命评估、诊断模型和报警策略参数。运维问答模块基于本地知识库和实时状态生成建议。

2.3 非功能需求分析

系统需要具备较好的可用性、可扩展性和可解释性。可用性方面，界面应直观展示关键指标，避免运维人员在多个页面之间频繁查找数据。可扩展性方面，系统应能够在后续接入真实 SCADA、CMS、油液监测和工单系统。可解释性方面，诊断结果不能只输出故障名称，还应说明风险因子、诊断依据、健康评分、RUL 和建议措施。

由于本系统是毕业设计原型，部署环境以本地演示为主，因此数据库选用 SQLite，能够满足轻量化部署要求。若后续进入实际风场应用，可迁移到 MySQL、PostgreSQL 或时序数据库，并增加消息队列、缓存和权限审计。

2.4 数据需求分析

项目数据来源包括两类。第一类是系统模拟采集数据，包含齿轮箱油温、高速轴承振动、油液颗粒度和有功功率。第二类是风机数据文件夹生成的数据摘要，包括自由报表、实时数据、故障统计和故障信息统计。自由报表包含风机名称、平均有功功率、平均风速、最大风速和环境温度；实时数据包含日期、发电机转速、有功功率、叶片角度、齿轮箱油温或齿轮箱加热、机舱位置等字段；故障统计包含故障天数、故障总数和峰值日期；故障信息统计包含故障码、出现次数、开始时间和结束时间。

系统根据这些数据形成机组快照，包括机组编号、运行状态、有功功率、平均功率、风速、最大风速、发电机转速、变桨角、油温、环境温度、机舱位置、振动 RMS、健康评分、RUL、故障数量、数据时间范围和数据文件来源。

2.5 需求约束与边界分析

本系统面向毕业设计和风电机组齿轮箱智能运维场景，系统需求既包括功能完整性，也包括工程可解释性。功能完整性要求系统能够围绕单台或多台华锐 SL1500 型双馈式风机完成运行状态展示、异常诊断、故障记录、报告生成和知识问答；工程可解释性要求每一个诊断结果都能够给出数据来源、特征依据、风险因子和维护建议，避免只输出一个难以理解的分标签。

在数据来源方面，系统当前采用模拟采集数据与风机统计数据文件相结合的方式。模拟采集模块用于持续生成油温、振动、油液、功率和数据质量等实时指标，风机数据仓库

用于读取风机汇总数据、故障统计和故障码标签。这样的设计能够在缺少真实在线 SCADA 接入条件下完成系统功能验证，同时保留将来接入真实 CMS、SCADA 和油液检测系统的接口边界。

在用户角色方面，系统至少需要区分管理员、运维人员和普通观察用户。管理员负责阈值配置、用户管理和系统参数维护；运维人员关注机组状态、故障诊断、报告和工单；普通观察用户主要查看风场总览和健康趋势。不同角色对数据写入、阈值修改和工单处理的权限不同，因此系统需要具备基本认证和权限控制能力。

在性能约束方面，系统页面需要满足普通浏览器流畅访问，实时监测数据刷新不能造成明显卡顿。对于毕业设计原型而言，重点不在高并发承载，而在接口组织清晰、数据刷新稳定、诊断结果可重复、页面交互完整。系统采用轻量级 SQLite 数据库和 Flask 服务，适合本地演示和小规模部署。

表 2.2 系统需求约束说明

约束类型	约束内容	设计处理
数据约束	真实风场在线数据不完整	采用模拟采集与风机统计文件融合，预留真实接口
场景约束	毕业设计演示环境以本地部署为主	使用 Flask、SQLite 和静态前端降低部署复杂度
算法约束	真实故障样本数量有限	采用规则库和风险评分，保留机器学习模型替换位置
可解释性	运维人员需要知道报警原因	输出特征值、风险因子、诊断依据和建议措施
扩展性	后续可能接入 SCADA/CMS/油液系统	按接口层、服务层和数据层分离设计

2.6 关键数据字段设计

齿轮箱健康管理涉及的数据字段较多，如果字段组织不清晰，后续的诊断计算、图表展示和报告生成都会变得混乱。因此本文将数据字段划分为实时状态字段、振动特征字段、油液状态字段、诊断结果字段和运维闭环字段五类。实时状态字段主要体现设备当前工况，振动特征字段用于故障诊断，油液状态字段用于评估润滑和磨损，诊断结果字段用于给出健康结论，运维闭环字段用于保存记录和处理状态。

在系统实现中，实时状态字段通常由数据采集服务生成或读取；振动特征字段由诊断模型计算得到；诊断结果字段由规则库、风险评分和 RUL 估计共同生成。前端页面不直接参与核心诊断计算，而是通过接口获取结构化结果并展示为仪表盘、表格、卡片和报告。这样的分工使前后端职责更加清晰，也便于后续单独替换诊断算法。

表 2.3 关键数据字段分类

字段类别	代表字段	作用
实时状态	unit_id、timestamp、oil_temp、power	描述机组当前运行工况
振动特征	rms、kurtosis、crest_factor、envelope_peak	识别冲击、磨损和啮合异常
油液状态	oil_quality、NAS、particle_level	反映润滑油污染和磨粒状态
诊断结果	fault_type、severity、health_score、rul_days	形成健康状态和剩余寿命判断
运维闭环	record_id、status、work_order、advice	支持故障记录、工单和维护建议

第3章 相关技术与理论基础

3.1 齿轮箱典型故障机理

风机齿轮箱故障可按照部件和机理分为齿轮故障、轴承故障、润滑故障、冷却故障、轴系故障和结构故障。齿轮故障包括齿面磨损、点蚀剥落、断齿和啮合异常。齿面磨损通常由润滑不足、载荷波动、异物颗粒和长期疲劳引起，会导致啮合频率附近能量升高；点蚀剥落属于接触疲劳损伤，常伴随冲击成分增强；断齿属于严重故障，可能出现周期性强冲击并引起快速停机风险。

轴承故障包括磨损、点蚀、剥落和高速轴承过温。轴承磨损初期可能表现为振动 RMS 缓慢升高，点蚀和剥落则会导致峭度、峰值因子和包络峰值升高。高速轴承过温可能与润滑不足、游隙异常、冷却效率下降或测点异常有关。

润滑和冷却故障会对齿轮箱整体健康产生连锁影响。润滑油污染或劣化会降低油膜承载能力，加速齿轮和轴承磨损；冷却系统故障会使油温升高，导致油液黏度变化和热变形增加。轴系不对中和箱体基础松动则通常表现为低频振动增强、宽频振动升高和工况变化下振动波动。

3.2 多源状态监测技术

风机齿轮箱状态监测通常需要融合 SCADA、CMS、油液检测和故障台账。SCADA 数据覆盖范围广，适合监测油温、功率、风速、转速、变桨角和机舱位置等低频工况信息。CMS 振动数据采样频率较高，适合分析齿轮啮合、轴承冲击和结构松动。油液检测能够反映磨粒、污染、水分和黏度变化。故障台账记录历史报警码和停机事件，可以为风险判断提供背景。

本文系统在代码中将采集测点抽象为 TEMP_GBX_OIL、VIB_HS_BRG、OIL_NAS 和 P_ACTIVE，分别代表齿轮箱油温、高速轴承振动、油液颗粒度和有功功率。系统同时读取风机数据摘要，将不同数据文件中的字段统一为机组快照，解决前端展示和诊断模块对数据格式的依赖问题。

3.3 振动特征提取方法

振动信号是机械故障诊断的重要依据。本文系统对振动信号提取以下特征：RMS 用于反映整体振动能量；峰值用于反映瞬时冲击强度；峭度用于识别脉冲冲击和局部损伤；峰值因子用于描述峰值相对于有效值的突出程度；脉冲因子用于描述峰值相对于平均绝对值的大小；包络峰值用于捕捉调制冲击特征。

系统中振动数据由正弦基频、轴频、啮合频率和随机噪声叠加生成，并在部分场景中加入冲击信号，以模拟轴承或齿面局部损伤。数据采集模块还使用移动平均方式进行滤波，减少随机噪声对波形展示和特征计算的影响。

3.4 风险评分与健康评估方法

表 3.1 齿轮箱健康评估主要风险因子

风险因子	含义	诊断作用
振动 RMS	反映整体振动能量	识别磨损、松动和运行异常
峭度	反映冲击尖峰程度	识别轴承剥落、齿面点蚀等早期冲击
峰值因子	峰值与有效值比值	增强对局部冲击的敏感性
包络峰值	反映调制冲击强度	辅助判断轴承和齿轮局部损伤
齿轮箱油温	反映热状态和冷却效率	识别过温和冷却系统异常
油液 NAS	反映润滑油污染程度	识别润滑劣化和磨粒风险

系统将多个特征和工况指标归一化为风险因子，包括振动 RMS、冲击峭度、峰值因子、包络峰值、齿轮箱油温和油液 NAS。每个风险因子根据经验阈值映射到 0 到 1 的区间，再采用加权求和得到综合风险值。振动 RMS 和冲击峭度权重较高，油温和油液状态作为重要补充。

健康评分由综合风险值和故障严重程度共同决定。风险越高，健康评分越低；若故障等级为警告或严重，则进一步扣减评分。RUL 根据健康评分、风险值和故障严重程度估计，健康评分高、风险值低时 RUL 较长；严重故障会显著缩短 RUL。虽然该方法属于规则型估计，但能够在毕业设计阶段形成可解释的健康管理流程。

3.5 Web 系统开发技术

系统采用 Flask 框架构建后端服务。Flask 具有轻量、灵活、易扩展的特点，适合毕业设计原型开发。数据库使用 Flask-SQLAlchemy 管理，表结构包括用户表、故障记录表、机组资产表和系统配置表。前端使用 HTML、CSS 和 JavaScript 实现，ECharts 用于趋势图、饼图和柱状图展示，Three.js 用于齿轮箱三维结构示意。系统通过 Server-Sent Events 实现实时状态推送，使页面能够持续刷新油温、振动和波形数据。

3.6 健康评分与 RUL 估计原理

健康评分是将多个监测指标压缩为一个便于运维人员理解的综合指标。本文系统并不直接使用单一阈值判断设备好坏，而是将振动 RMS、冲击峭度、峰值因子、包络峰值、齿轮箱油温和油液 NAS 等级归一化后进行加权计算。这样可以避免某个指标轻微波动就触发严重报警，也能在多个指标同时升高时提高风险敏感性。

RUL 估计用于描述设备在当前退化状态下的剩余可运行时间。由于毕业设计原型缺少长期真实退化样本，本文采用基于健康评分、风险因子和故障严重度的启发式估计方法。该方法不是工业现场最终寿命模型，但能够展示健康管理系统中寿命预测的接口形式和计算流程。后续如果获得真实故障样本，可以将该位置替换为 SVR、随机森林、LSTM 或 Transformer 等预测模型。

在健康评分计算中，油温和振动代表齿轮箱当前运行负荷与机械冲击状态，油液 NAS 等级代表润滑和磨粒情况，历史故障记录代表设备长期退化背景。系统将这些因素综合为风险分数，再将风险分数映射为健康评分。健康评分越低，系统越倾向于给出缩短复检周期、安排停机检查或创建检修工单的建议。

在 RUL 估计中，系统将健康评分作为基础寿命因子，将风险分数作为惩罚项，将故障严重度作为额外惩罚项。例如严重故障会显著降低 RUL 估计值，普通告警只会适当缩短复检周期。这样设计的优点是计算逻辑清晰、结果可解释，适合毕业设计阶段展示智能健康管理体系的完整闭环。

表 3.1 健康评估指标含义

指标	含义	异常说明
振动 RMS	反映整体振动能量	升高可能表示磨损、松动或不对中
峭度	反映冲击尖峰程度	升高可能表示轴承点蚀或齿面剥落
峰值因子	峰值与有效值之比	升高表示局部冲击增强
包络峰值	捕捉调制冲击特征	适合识别轴承和齿轮早期故障
油温	反映热状态和冷却能力	升高可能与冷却、润滑或负荷有关
油液 NAS	反映颗粒污染程度	升高说明油液污染或磨粒增多

3.7 智能问答与知识库技术

智能问答模块的目的不是替代诊断模型，而是提升系统的人机交互能力。运维人员面对油温过高、振动异常、油液污染等问题时，往往需要查阅手册、经验记录和故障案例。系统将这些知识整理为本地知识库，并结合当前机组状态生成回答，使用户能够通过自然语言方式获得排查建议。

知识库条目包括故障现象、可能原因、检测方法、处理建议和复检周期等内容。系统收到问题后，先根据关键词和意图匹配知识条目，再结合实时状态数据补充当前工况。例如用户询问齿轮箱油温过高时，系统会同时考虑当前油温、振动 RMS、油液等级和功率负荷，从而给出更接近现场的排查顺序。

该模块也预留了兼容 OpenAI 格式的大模型接口。如果配置 API Key，系统可以把本地知识和实时状态作为上下文，生成更完整的解释；如果接口不可用，则回退到本地专家规则。这样的设计兼顾了演示稳定性和技术扩展性，避免系统过度依赖外部服务。

第 4 章 系统总体设计

4.1 系统架构设计

系统功能架构如图 4.2 所示。系统按表现层、接口层、业务服务层和数据层组织，前端负责状态展示与交互操作，后端负责数据处理、诊断计算、报告生成和知识问答，数据层为诊断模型和页面展示提供基础支撑。

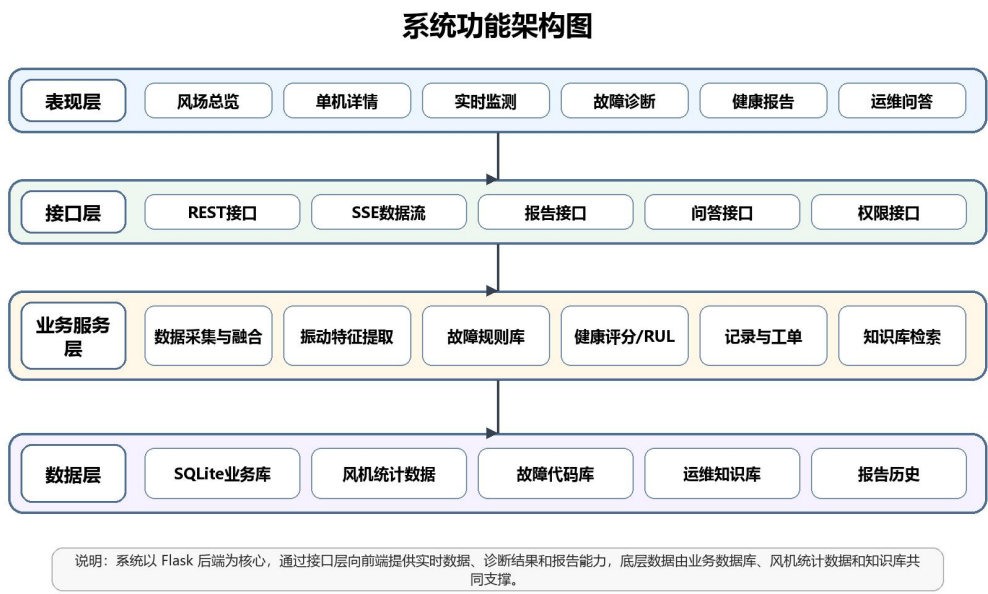


图 4.2 系统功能架构图

系统总体采用 B/S 架构，用户通过浏览器访问前端页面，前端通过 HTTP 接口和 SSE 通道与 Flask 后端通信。后端由应用入口、路由层、服务层、数据模型层和数据仓库层组成。应用入口负责创建 Flask 应用、配置数据库和注册蓝图；路由层提供认证、数据、风场、报告、问答、设置和运维接口；服务层负责数据模拟、数据融合、诊断计算和知识问答；数据模型层负责数据库表定义；数据仓库层负责读取风机数据摘要文件。

前端按业务区域划分为健康监测区、智能诊断区和系统配置区。健康监测区包括风场总览、单机详情和实时监测；智能诊断区包括故障诊断、运维辅助问答、故障代码库、故障数据和健康报告；系统配置区包括参数设置。另有独立 HMI 页面用于展示单机齿轮箱人机交互界面。

4.2 数据库设计

表 4.1 数据库表结构说明

数据表	关键字段	用途
User	username、password、role	保存用户账号、密码哈希和角色
FaultRecord	unit_id、timestamp、fault_type、severity、probability、status	保存诊断结果和处理状态

TurbineAsset	unit_id、number、model、design_life_years	保存风机资产基础信息
SystemConfig	config_key、config_value、description	保存油温、振动、油液等阈值配置

系统数据库采用 SQLite。User 表保存用户信息，包括用户名、密码和角色；FaultRecord 表保存诊断记录，包括机组编号、时间戳、故障类型、严重程度、置信度、建议措施和处理状态；TurbineAsset 表保存机组资产信息，包括机组编号、数字编号、型号、位置、状态、设计寿命和投运日期；SystemConfig 表保存系统参数，包括配置键、配置值和描述。

该数据库设计虽然较轻量，但能够覆盖毕业设计原型的核心数据需求。用户表支撑登录和权限展示；故障记录表支撑诊断结果落库、列表查看和 CSV 导出；机组资产表为后续真实风场资产管理预留空间；系统配置表用于保存油温、振动、油液等阈值参数。

4.2.1 数据库表关系说明

数据库设计围绕用户、故障记录、参数配置和闭环处理展开。User 表用于保存用户账号、密码哈希和角色信息，支撑登录与权限控制；FaultRecord 表保存诊断产生的故障类型、严重程度、置信度、处理状态和建议措施；FaultClosure 表记录故障闭环过程，包括处理人、处理说明、复测结果和关闭时间；配置表则保存油温、振动、油液等阈值。

在业务流程上，诊断模块生成结果后会写入故障记录表，前端故障数据页面从该表读取记录并展示；当运维人员完成处理后，系统将处理信息写入闭环表，并更新故障记录状态。健康报告模块读取实时状态、历史故障和闭环记录，生成面向运维人员的综合报告。这样的数据库关系能够支持从报警发现到处理反馈的完整闭环。

表 4.2 数据库表关系说明

数据表	主要字段	业务作用
User	id、username、password_hash、role	保存用户和角色权限
FaultRecord	unit_id、fault_type、severity、status	保存诊断结果和处理状态
FaultClosure	record_id、handler、action、result	保存故障闭环处理信息
SystemConfig	threshold_name、value、updated_at	保存阈值和系统参数
ReportHistory	report_id、unit_id、summary、created_at	保存健康报告摘要

4.3 后端接口设计

表 4.2 系统主要后端接口

接口类别	典型接口	功能说明
数据接口	/api/data/status、/api/data/diagnosis、 /api/data/trend	状态采集、诊断计算和趋势分析

风场接口	/api/windfarm/overview、 /api/windfarm/turbine/<unit_id>	风场总览和单机详情
报告接口	/api/report/generate、 /api/report/health-summary	生成健康报告和摘要
问答接口	/api/qa/ask	运维知识问答和实时状态分析
运维接口	/api/ops/workorders、/api/ops/alarm-policy	工单模拟、报警策略和模型验证

系统后端接口按蓝图划分。数据接口 /api/data 提供振动波形、实时状态、故障代码、诊断结果、故障记录、故障记录导出、趋势分析和 SSE 数据流。风场接口 /api/windfarm 提供风场总览、单机详情、参数分组和风场流式数据。报告接口 /api/report 提供健康报告和报告历史。问答接口 /api/qa 提供运维问题回答。运维接口 /api/ops 提供 SCADA 状态、工单、模型验证、审计日志、趋势对比和报警策略。认证和设置接口分别负责登录注册和系统配置。

这种接口组织方式使系统业务边界较清楚。实时监测和诊断相关功能集中在数据接口中，风场和单机展示集中在风场接口中，报告和问答作为独立能力存在，便于后续扩展。

4.3.1 接口设计原则

后端接口按业务域划分为认证接口、数据接口、风场接口、报告接口、问答接口、设置接口和运维接口。认证接口负责登录、注册和当前用户信息；数据接口负责实时状态、振动数据、故障诊断、故障记录和故障代码库；风场接口负责风场总览和单机详情；报告接口负责生成健康报告；问答接口负责运维知识问答；设置接口负责阈值和用户管理；运维接口负责工单和报警策略。

接口返回数据尽量采用统一的 JSON 结构，前端页面只关注字段含义，不关心后端内部计算过程。这样一方面可以降低前端复杂度，另一方面也便于后续把规则诊断模块替换为机器学习服务或独立微服务。接口设计中还保留了 SSE 数据流，用于实时推送状态变化，增强监测页面的实时感。

表 4.3 后端接口分组说明

接口分组	代表路径	功能说明
认证接口	/api/auth/login	用户登录和身份验证
数据接口	/api/data/status	实时状态、振动和诊断数据
风场接口	/api/windfarm/overview	风场总览和单机详情
报告接口	/api/report/generate	生成健康报告
问答接口	/api/qa/ask	运维知识问答
运维接口	/api/ops/workorders	工单模拟和闭环处理

4.4 诊断流程设计

齿轮箱智能健康管理流程

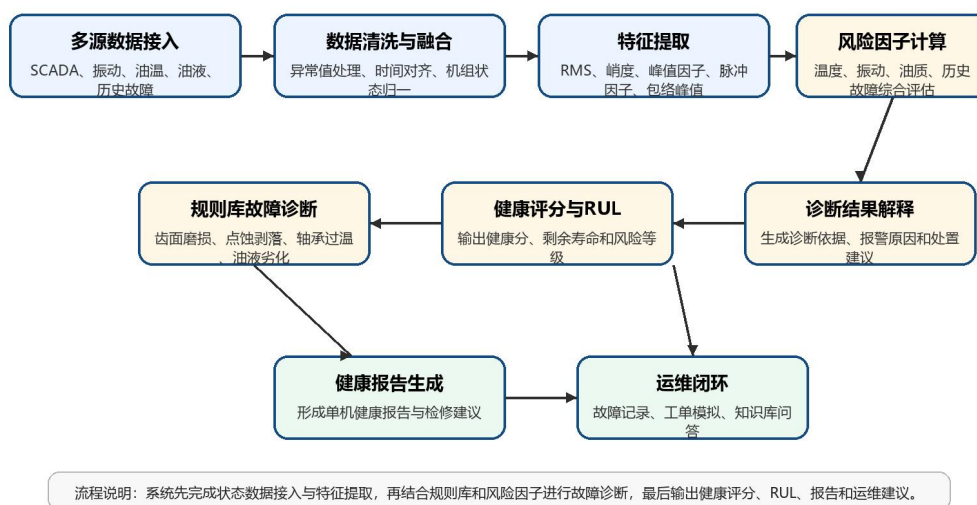


图 4.1 齿轮箱智能健康管理流程图

系统诊断流程如图 4.1 所示。该流程以多源状态数据为输入，经过数据清洗、特征提取、风险因子计算和规则库诊断后，形成健康评分、RUL 估计、诊断依据与运维建议，并通过健康报告、故障记录和工单模拟实现闭环管理。

系统诊断流程从选择机组和诊断场景开始。后端获取当前机组状态，生成或读取振动波形，并根据场景对油温、振动 RMS、油液质量和原始波形进行调整。例如过温场景会提高油温，不对中场景会叠加低频振动，轴承点蚀场景会加入周期性冲击，断齿场景会加入更强冲击并提高振动和油液风险。

随后系统提取振动特征并计算风险因子。综合风险值输入故障分类规则，得到故障类型、关联部件、严重程度和诊断依据。系统进一步计算健康评分和 RUL，生成维护动作清单。若诊断结果为警告或严重，系统模拟生成检修工单信息，包括工单编号、优先级、责任班组、建议处置窗口和闭环流程。最后，诊断记录写入数据库，供故障数据管理页面查看和导出。

4.5 前端页面设计

前端首页包含登录界面和主应用界面。登录后默认进入健康总览，左侧导航按照健康监测区、智能诊断区和系统配置区分组，顶部显示当前模块说明和通知。风场总览页面展示多台机组卡片，单机详情页面展示齿轮箱温度、寿命、部件状态、告警和集群对标。实时监测页面展示油温、振动、功率、RUL、采集链路、振动波形和数字孪生热图。

故障诊断页面提供场景选择和诊断按钮，运行后展示疑似故障类型、风险等级、关联部件、置信度、健康评分、RUL、特征值、风险因子、诊断依据、建议措施和工单信息。故障数据页面展示历史故障记录，报告页面展示健康评估结果，问答页面支持输入自然语言问题并获得排查建议。HMI 页面则以单机齿轮箱为对象，提供运行总览、状态寿命、透明模型、数据采集、测点信号、阈值设置、故障代码和临界警报等功能。

第 5 章 系统详细实现

5.1 数据采集与融合模块实现

数据采集模块位于 `services/data_acquisition.py`。该模块定义了四类传感器：齿轮箱油温、高速轴承振动、油液颗粒度和有功功率。系统通过基础值、周期波动和随机噪声生成测点数据，并通过多次采样和均值滤波降低异常噪声影响。采集结果包括油温、振动 RMS、油液 NAS、功率、链路延迟、丢包率、采样率、采集状态、数据质量、在线传感器数量、健康评分、RUL 和告警列表。

代码清单 5.1 振动信号特征提取核心代码

```
def process_signals(signal_data):
    arr = np.array(signal_data, dtype=float)
    rms = float(np.sqrt(np.mean(arr ** 2)))
    peak = float(np.max(np.abs(arr)))
    mean = float(np.mean(arr))
    std = float(np.std(arr)) or 1e-6
    centered = arr - mean
    kurtosis = float(np.mean(centered ** 4) / (std ** 4))
    crest_factor = float(peak / rms) if rms > 0 else 1.0
    impulse_factor = float(peak / (np.mean(np.abs(arr)) or 1e-6))
    envelope = get_envelope(arr.tolist())
    envelope_peak = float(max(envelope)) if envelope else 0.0
    return {
        "rms": round(rms, 4),
        "kurtosis": round(kurtosis, 4),
        "peak": round(peak, 4),
        "crest_factor": round(crest_factor, 4),
        "impulse_factor": round(impulse_factor, 4),
        "envelope_peak": round(envelope_peak, 4),
    }
```

风机数据仓库模块位于 `services/wind_data_repository.py`。该模块读取风机数据 `/wind_data_summary.json`，将自由报表、实时数据、故障统计和故障信息统计整合为统一机组快照。对于每台机组，系统计算功率、风速、发电机转速、齿轮箱油温、环境温度、振动 RMS、健康评分和 RUL。对于历史故障，系统根据故障码映射中文标签，并将故障码分为齿轮箱关联故障、运行事件和数据采集故障等类别。

系统运行后的风场健康总览页面如图 5.1 所示。该页面以矩阵形式展示多台机组的运行状态，并在顶部汇总时间可利用率、能量可利用率、平均油温、总有功功率、平均健康评分和告警机组数量。



图 5.1 风场健康总览页面

风场总览页面是运维人员进入系统后最先接触的页面，其设计目标是快速定位异常机组。页面中不同颜色代表不同运行状态，绿色表示正常运行，橙色和红色表示告警或故障，紫色表示维护停机。每个机组卡片展示功率、风速、油温、健康评分和运行状态，便于用户在一个页面中完成多机组横向比较。

系统还提供指标列表和健康矩阵两种浏览方式。健康矩阵适合快速扫描，指标列表适合查看详细数值。运维人员可以先通过矩阵发现异常机组，再进入单机详情页面查看趋势、故障代码和检修建议。该设计符合风电场运维中“先总览、再下钻、后处理”的工作习惯。

5.2 故障诊断算法模块实现

故障诊断模块位于 `services/ml_models.py`。系统首先调用 `process_signals` 计算振动特征，包括 RMS、峭度、峰值、峰值因子、脉冲因子和包络峰值。包络估计通过信号和梯度构造近似解析信号，再进行滑动平均，用于捕捉冲击调制特征。

代码清单 5.2 故障诊断与健康评分调用逻辑

```
def run_diagnosis(unit_id="WTG-001", raw_signal=None, status=None):
    raw_signal = raw_signal or generate_demo_signal()
    features = process_signals(raw_signal)
    risk_score, risk_factors, snapshot = score_risk(features, status)
    fault = classify_fault(features, risk_score, risk_factors, status)
    health_score = calculate_health_score(risk_score, fault["severity"])
    rul = predict_rul(health_score, risk_score, fault["severity"])
    explanation = build_explanation(
        features, fault["severity"], rul, risk_score, risk_factors, fault
    )
    return {
```

```

    "unit_id": unit_id,
    "features": features,
    "fault_type": fault["type"],
    "health_score": health_score,
    "rul_days": rul,
    "explanation": explanation,
}

```

随后系统调用 `score_risk` 计算风险因子。风险因子包括振动 RMS、冲击峭度、峰值因子、包络峰值、齿轮箱油温和油液 NAS。系统采用归一化函数将原始值映射到 0 到 1，再按权重计算综合风险值。分类函数 `classify_fault` 根据强制场景、油温风险、油液质量、冲击特征和振动特征识别故障类型。系统故障库覆盖正常运行、齿轮齿面磨损、齿面点蚀/剥落、齿轮断齿、轴承磨损、轴承点蚀/剥落、高速轴承过温、齿轮箱油温过高、润滑油污染/油液劣化、轴系不对中、齿轮啮合异常、箱体或基础松动和冷却系统故障。

健康评分由 `calculate_health_score` 生成，RUL 由 `predict_rul` 估计。系统还会调用 `build_explanation` 生成诊断说明，包括风险等级、风险分数、结论、诊断依据、主要风险因子、特征含义和 RUL 说明。最终 `run_diagnosis` 将所有结果封装为接口响应，并尝试写入 `FaultRecord` 数据表。

故障诊断页面如图 5.2 所示。用户可以选择机组和诊断场景，系统执行诊断后输出故障类型、风险等级、健康评分、RUL、诊断依据和建议措施。



图 5.2 故障诊断页面

故障诊断页面是系统的核心业务页面。用户可以选择不同机组，也可以选择正常、齿轮磨损、点蚀剥落、轴承过温、油液污染、冷却故障等场景进行演示。点击开始诊断后，前端调用后端诊断接口，后端完成振动特征提取、风险评分、故障分类和结果解释，然后

将结构化结果返回页面展示。

诊断结果并不只显示故障名称，而是同时给出风险等级、部件位置、置信度、健康评分和剩余寿命。下方的诊断依据和建议措施用于解释为什么系统给出该结论，以及下一步应该如何排查。这样的展示方式能够增强诊断结果可信度，避免用户只看到一个缺少依据的告警标签。

5.3 风场总览与单机详情实现

风场总览接口位于 `routes/windfarm_route.py`。当风机数据摘要文件存在时，系统优先使用真实文件数据；否则使用模拟数据。总览结果包括风场名称、系统名称、时间戳、可利用率、平均温度、平均风速、总功率、平均健康评分、告警数量、运行机组数量、机组总数、状态统计和机组列表。

单机详情接口 `/api/windfarm/turbine/<unit_id>` 返回指定机组的运行指标、温度、寿命、部件状态、集群对标、控制模式、SCADA 时间和告警列表。寿命部分包括设计寿命、已运行年限、剩余设计年限、齿轮箱剩余寿命、健康评分、RUL、复检周期和复检建议。部件状态包括低速轴承、高速轴承、齿轮箱油温、振动 RMS 和润滑油。

故障数据管理页面如图 5.3 所示。该页面用于查看诊断记录、筛选故障状态、导出 CSV 并进入故障闭环处理。

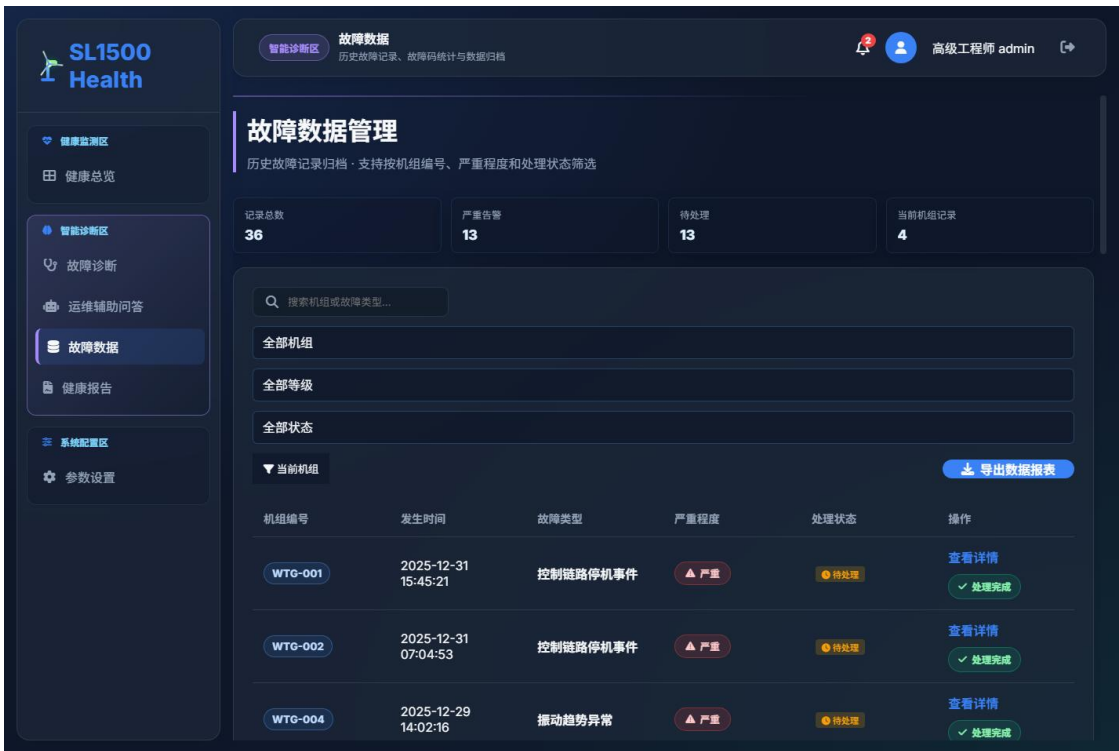


图 5.3 故障数据管理页面

故障数据管理页面承担历史记录追踪功能。系统每次诊断产生的记录会写入数据库，页面按机组、严重程度、处理状态和关键词进行筛选。对于严重或待处理故障，运维人员

可以进入详情查看诊断依据、建议措施和处理进度。

该页面还提供 CSV 导出功能，便于将故障记录用于论文测试、运维日报或后续数据分析。对于毕业设计而言，故障记录页面能够证明系统不是只做一次性诊断，而是具备数据积累和闭环管理能力。

5.4 故障代码库与数据管理实现

故障代码库位于 `routes/data_route.py`。系统内置 15 个齿轮箱关联故障项，如齿轮箱油温过高、齿轮箱油温趋势异常、高速轴承温度异常、振动 RMS 超限、齿轮啮合异常、齿面点蚀/剥落、齿轮断齿风险、润滑油污染/劣化、冷却系统故障、轴系不对中、箱体/基础松动和 RUL 低于预警阈值等。每个故障项包含故障码、名称、类别、关联信号、单位、关注阈值、临界阈值、来源和建议措施。

系统还根据历史故障信息统计动态生成现场故障码。对于不在齿轮箱内置映射中的故障码，系统生成 SCADA- 前缀的故障项，并根据标签判断故障类别和严重程度。故障码状态根据当前值与阈值判断为正常、关注、临界或未知。

故障数据管理通过 `FaultRecord` 表和风机文件故障记录共同实现。诊断模块产生的记录会写入数据库；若风机数据文件存在，系统也会根据历史故障信息生成记录列表。系统支持 CSV 导出，导出字段包括机组编号、发生时间、故障类型、严重程度、置信度、处理状态、工单动作和建议措施。

健康报告页面如图 5.4 所示。系统根据当前机组状态和历史故障统计生成健康评分、关键指标、诊断结论和维护建议。



图 5.4 健康报告页面

健康报告页面将实时监测和故障诊断结果转化为面向运维管理的文档化信息。报告中

包括机组编号、统计范围、生成时间、健康评分、油温、振动 RMS、油液 NAS、功率、数据质量、RUL 和复检周期等关键指标。相比单纯的仪表盘，健康报告更适合用于阶段性检查和汇报。

报告结论部分会根据健康评分和风险因子生成文字说明，指出当前机组的主要风险和维护建议。如果系统发现油温、振动或油液指标异常，报告会提示缩短复检周期或安排现场检查。该功能体现了智能健康管理体系从监测数据到管理决策的转化能力。

5.5 AI 运维问答模块实现

AI 运维问答模块位于 `services/rag_service.py` 和 `routes/qa_route.py`。系统内置本地知识库，知识项包括油温过高排查、高速轴承冲击诊断、齿面磨损与点蚀、齿轮断齿与啮合异常、轴承磨损与高速端过温、润滑油污染、联轴器不对中、箱体或基础松动、齿轮箱故障清单、剩余寿命评估、M-IALO-SVR 方法和检修策略。

代码清单 5.3 前端实时状态刷新逻辑

```
async function refreshStatus() {
  const response = await fetch("/api/data/status");
  const status = await response.json();
  document.querySelector("#oilTemp").textContent = status.oil_temp + " °C";
  document.querySelector("#vibrationRms").textContent =
    status.vibration_rms + " mm/s";
  healthChart.setOption({
    series: [{ data: [{ value: status.health_score, name: "健康评分" }] }]
  });
}
setInterval(refreshStatus, 5000);
refreshStatus();
```

当用户提出问题时，系统先识别问题意图，如故障分类、算法原理、排查建议、原因分析、寿命预测或状态评估。然后根据关键词和内容匹配知识库条目，并结合当前油温、振动、油液、功率、健康评分和 RUL 生成结论、实时工况、判断依据、建议步骤和可继续追问。如果开启大模型辅助分析且配置了 API Key，系统会调用兼容 OpenAI 格式的聊天接口生成更完整回答；若调用失败，则回退到本地专家引擎。

AI 运维问答页面如图 5.5 所示。用户可以围绕油温、振动、轴承、齿轮、润滑和检修策略提出问题，系统结合知识库和当前状态生成回答。



图 5.5 AI 运维问答页面

AI 运维问答页面面向运维人员的日常咨询场景。传统系统通常只能提供固定报警，而问答模块可以让用户以自然语言提出问题，例如“齿轮箱油温过高如何排查”“振动 RMS 升高是否需要停机”“油液 NAS 等级偏高如何处理”等。系统根据问题意图匹配知识库条目，并结合当前状态给出排查顺序。

在实现上，问答模块先使用本地知识库保证回答稳定性，再根据配置决定是否调用外部大模型接口。即使没有网络或 API Key，系统仍然能够返回专家规则答案。这样既满足毕业设计演示稳定性，也体现了智能运维系统可扩展到大模型辅助分析的方向。

5.6 健康报告与工单闭环实现

健康报告接口位于 `routes/report_route.py`。系统根据当前机组状态、历史趋势和最新故障记录生成报告。报告内容包括报告编号、机组编号、统计周期、生成时间、健康评分、摘要、关键指标、诊断结果、维护计划和工单动作。当接入风机数据文件时，报告会展示数据范围、平均油温、振动峰值、油液 NAS、有功功率、数据质量、RUL、复检周期、报警级别和故障记录数量。

工单模拟接口位于 `routes/ops_route.py`。当诊断结果为警告或严重时，系统可生成 P2 或 P1 工单，包含工单编号、机组编号、故障类型、优先级、阶段、责任人、创建时间和闭环流程。闭环流程包括诊断触发、生成工单、现场复核、检修处理和复测验收。

系统参数设置页面如图 5.6 所示。该页面用于配置油温、振动、油液阈值和系统访问参数，阈值变化会影响实时告警和健康报告。



图 5.6 系统参数设置页面

系统设置页面用于体现健康管理体的可配置性。不同风场、不同季节和不同运行策略下，油温、振动和油液阈值可能需要调整。如果阈值完全写死在代码中，系统很难适应现场环境。因此系统提供参数配置页面，使管理员能够根据运维策略调整预警阈值和严重阈值。

参数配置不仅影响页面显示，也会影响诊断结果、告警等级和报告建议。这样可以保证系统的业务逻辑与管理策略一致。用户管理和角色权限则用于控制不同人员的操作范围，避免普通用户误改关键阈值。

单机齿轮箱详情页面如图 5.7 所示。页面从风场总览下钻到单台机组，集中展示齿轮箱油温、高速轴承温度、振动 RMS、油液 NAS 等级、健康评分、RUL、复检周期和历史故障记录。



图 5.7 单机齿轮箱详情页面

单机详情页面体现了系统从风场级监控到设备级诊断的下钻能力。运维人员在风场总览中发现异常机组后，可以进入该页面查看机组的关键指标和部件状态。该页面还提供 HMI 控制台、故障诊断、实时趋势和健康报告入口，使单台机组相关功能形成集中工作台。

从健康管理角度看，单机详情页面能够把不同来源的数据组织到同一个设备视图中。油温和轴承温度用于判断热状态，振动 RMS 用于判断机械冲击，油液 NAS 用于判断润滑污染，RUL 和复检周期用于形成维护计划。这样的页面结构有助于运维人员快速理解设备状态。

齿轮箱 HMI 页面如图 5.8 所示。该页面面向现场监控场景，用更加接近工业界面的方式展示单机齿轮箱状态和关键指标。

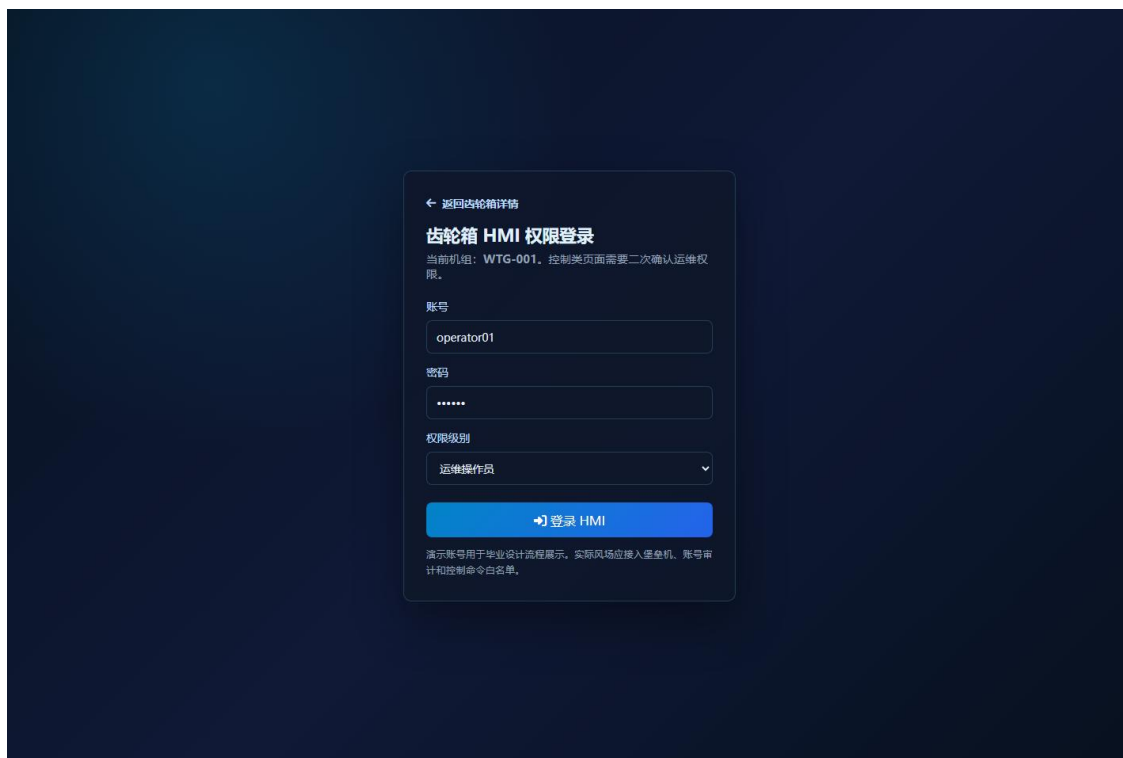


图 5.8 齿轮箱 HMI 人机交互页面

HMI 页面主要用于展示齿轮箱关键运行信息和人机交互状态。与普通 Web 仪表盘相比，HMI 页面更强调现场监控感和设备对象感，适合在答辩演示时说明系统具备工业监控界面的扩展能力。页面可根据机组编号加载不同设备状态，并与后端接口保持一致的数据来源。

通过风场总览、单机详情和 HMI 页面的组合，系统形成了三层展示结构：风场总览用于宏观监控，单机详情用于设备分析，HMI 页面用于现场交互展示。这种结构较符合风电场运维从场站到机组再到部件的业务层级。

第 6 章 系统测试与结果分析

6.1 测试环境

系统测试环境为 Windows 本地环境，后端使用 Python 虚拟环境运行 Flask 服务，浏览器访问 `http://127.0.0.1:5000`。后端依赖包括 Flask、Flask-CORS、Flask-SQLAlchemy、NumPy、Pandas、scikit-learn、python-dotenv、xlrd 和 OpenAI 兼容库等。前端依赖 ECharts、Three.js 和 Font Awesome。

6.2 功能测试

表 6.1 系统功能测试结果

测试项目	测试内容	结果
登录注册	管理员登录、新用户注册和角色显示	通过
风场总览	多机组状态、统计指标和机组卡片展示	通过
单机详情	切换机组后关键指标同步变化	通过
实时监测	SSE 推送油温、振动、功率和波形数据	通过
故障诊断	不同故障场景输出诊断结果和建议	通过
报告导出	生成健康报告并导出故障记录 CSV	通过
运维问答	基于知识库和当前工况生成排查建议	通过

登录注册测试中，系统能够完成管理员账号登录和新用户注册，并在前端显示用户角色。风场总览测试中，系统能够展示多机组运行状态、风场统计指标和状态图例。单机详情测试中，选择不同机组后，油温、功率、风速、健康评分、RUL、告警和集群对标能够随之变化。

实时监测测试中，系统通过 SSE 持续推送状态数据和振动波形，前端能够刷新油温、振动、功率、RUL、采集链路和图表。故障诊断测试中，选择不同场景后，系统能够生成相应故障类型、风险等级、置信度、健康评分、RUL、风险因子和维护建议。故障数据测试中，系统能够显示诊断记录和历史故障记录，并支持导出 CSV。

故障代码库测试中，系统能够根据当前值和阈值判断故障项状态，并支持按机组展示。健康报告测试中，系统能够生成报告摘要和关键指标。AI 问答测试中，系统能够回答油温过高、振动异常、轴承故障、齿轮故障、油液污染、检修周期和算法原理等问题。

6.3 诊断场景测试分析

系统提供多种诊断场景。正常运行场景中，油温、振动和油液质量均处于较低风险范围，系统输出正常运行，健康评分较高，维护建议为保持日常巡检。齿轮齿面磨损场景中，振动能量和油液污染风险轻度升高，系统输出齿轮齿面磨损，并建议 72 小时内复测振动趋势和取油样复核。

齿面点蚀/剥落和齿轮断齿场景中，系统通过加入周期性冲击提高峭度、峰值因子和包络峰值，输出严重风险，并建议降载或停机检查。轴承点蚀/剥落场景中，冲击特征明显升高，系统建议结合 120-240Hz 特征频段和包络谱进行复核。油温过高和冷却系统故障场景中，油温风险因子升高，系统建议检查散热器、油冷风扇、三通阀、温控开关、油位和滤芯压差。

油液污染场景中，油液 NAS 指标升高，系统建议取样复测 NAS 等级、水分、黏度和金属颗粒，并检查滤芯、呼吸器和密封。轴系不对中和基础松动场景中，低频振动增强，系统建议复核联轴器、地脚螺栓、弹性支撑和基础连接。

6.4 结果评价

从功能完整性看，系统覆盖了毕业设计要求中的登录、系统管理、用户管理、故障数据、数据分析、参数设置、故障分析、AI 智能问答和知识库等内容。从工程流程看，系统能够形成从数据采集、特征提取、诊断判断、健康评分、RUL 估计、建议生成、记录保存到报告和工单的闭环。从展示效果看，系统页面较适合风电运维场景，具有多机组总览、单机下钻、图表分析和 HMI 展示能力。

系统不足也较明显。当前诊断算法主要是规则模型和模拟信号，尚未完成真实 M-IALO-SVR 模型训练和大规模故障样本验证。前端展示中提到的 M-IALO-SVR、VMD-CNN、包络阶次分析和小波散度等内容有一定演示性质，正式论文答辩时需要说明系统当前处于原型阶段，并强调已预留真实模型接入位置。此外，系统尚未进行高并发、长时间运行和生产安全审计测试。

6.5 测试用例设计

系统测试围绕功能正确性、接口稳定性、页面展示效果和诊断结果合理性展开。由于本系统是毕业设计原型，测试重点不是高并发压力，而是验证主要业务流程是否完整：用户能否登录，风场总览是否正常展示，诊断接口是否返回结构化结果，故障记录是否保存，报告是否生成，问答模块是否能够给出合理建议，阈值配置是否能够影响后续判断。

测试过程中采用黑盒测试与接口检查相结合的方法。黑盒测试从用户角度操作页面，观察页面元素、图表和表格是否正常；接口检查则直接访问后端接口，确认返回字段是否完整。对于故障诊断场景，系统分别测试正常运行、油温过高、轴承冲击、油液污染、齿轮啮合异常和冷却系统故障等典型状态。

表 6.2 功能测试用例

测试编号	测试内容	预期结果	测试结论
TC-01	用户登录	输入正确账号后进入系统主页	通过
TC-02	风场总览	显示机组矩阵和关键统计指标	通过
TC-03	故障诊断	返回故障类型、健康评分和建议措施	通过
TC-04	故障记录	诊断结果写入记录并可筛选查看	通过

TC-05	健康报告	生成机组健康指标和文字结论	通过
TC-06	AI 问答	根据问题返回排查建议	通过
TC-07	参数设置	阈值可修改并影响告警策略	通过

6.6 典型诊断场景结果

为了验证诊断逻辑的合理性，本文选取正常运行、齿轮箱油温过高、轴承冲击、润滑油污染和齿轮啮合异常五类场景进行对比测试。每个场景通过不同的油温、振动 RMS、油液 NAS 和振动冲击特征组合触发。系统根据风险因子计算结果给出不同的健康评分和维护建议。

正常运行场景中，油温、振动和油液指标均处于较低水平，系统输出健康评分较高，建议保持日常巡检。油温过高场景中，系统重点提示冷却风扇、油冷器、油位和滤芯压差检查。轴承冲击场景中，峭度和包络峰值升高，系统提示复核高速轴承包络谱。油液污染场景中，NAS 等级升高，系统建议取样复检和检查滤芯。齿轮啮合异常场景中，振动能量和峰值因子同步升高，系统提示检查齿面接触斑和啮合频率。

表 6.3 典型诊断场景测试结果

场景	主要异常指标	系统诊断	维护建议
正常运行	指标均正常	正常运行	保持日常巡检
油温过高	oil_temp 升高	齿轮箱油温过高	检查冷却系统和润滑状态
轴承冲击	kurtosis、envelope_peak 升高	轴承点蚀/剥落	复核包络谱并安排检查
油液污染	NAS 等级升高	润滑油污染/劣化	取样复检并检查滤芯
啮合异常	RMS 和峰值因子升高	齿轮啮合异常	检查齿面和载荷波动

6.7 系统运行效果分析

从运行效果看，系统能够将风场级总览、单机级详情、诊断级解释和报告级输出连接起来。风场总览用于发现异常对象，故障诊断用于判断异常类型，故障记录用于保存历史，健康报告用于形成汇总结论，AI 问答用于补充排查知识，参数设置用于调整策略。各模块之间不是孤立页面，而是围绕齿轮箱健康管理形成了较完整的业务链条。

从交互效果看，系统页面采用侧边导航和分区显示方式，运维人员可以在健康监测区、智能诊断区和系统配置区之间切换。关键指标采用卡片和图表展示，故障记录采用表格和筛选器展示，报告页面采用文档化结构展示。整体交互符合运维系统需要快速浏览、快速定位和快速处理的特点。

从工程实现看，系统后端采用蓝图拆分接口，前端采用模块化函数组织页面逻辑，数

数据库保存用户和故障记录。虽然系统仍属于毕业设计原型，但已经具备从数据生成、特征提取、诊断判断、结果解释、记录保存到报告生成的基本闭环。后续如果接入真实风场数据，系统可以在现有接口基础上继续扩展。

6.8 单机详情与 HMI 验证

单机详情与 HMI 页面测试主要验证系统的下钻分析能力。测试时从风场总览页面选择 WTG-001 机组，系统能够进入齿轮箱详情页面，并显示齿轮箱油温、高速轴承温度、振动 RMS、润滑油 NAS 等级、健康评分、剩余寿命和历史故障记录等信息。页面按钮能够跳转到 HMI、故障诊断、实时趋势和健康报告，说明页面之间的业务关联能够正常运行。

HMI 页面测试采用直接访问和详情页跳转两种方式。测试结果表明，HMI 页面能够按照指定机组加载显示内容，页面布局稳定，关键指标显示清晰。虽然当前 HMI 页面仍属于演示型界面，但它能够说明系统具备面向现场监控终端扩展的可能性，为后续接入真实 PLC、SCADA 网关或工业组态界面提供了展示基础。

表 6.4 页面截图验证结果

页面	验证内容	结果
风场总览	机组矩阵、统计指标、状态颜色	显示正常
单机详情	油温、振动、RUL、复检周期	显示正常
故障诊断	场景选择、诊断结果、建议措施	显示正常
故障记录	记录列表、筛选、导出入口	显示正常
健康报告	关键指标、结论、报告结构	显示正常
AI 问答	问题输入、知识回答、上下文展示	显示正常
参数设置	阈值配置、角色和用户管理	显示正常
HMI 页面	工业监控界面和机组状态	显示正常

6.9 非功能测试与可用性分析

非功能测试主要关注系统的稳定性、可用性、可维护性和扩展性。稳定性方面，系统在本地 Flask 服务环境下能够连续刷新数据并完成多次诊断请求，页面没有出现明显卡死。可用性方面，侧边栏按健康监测区、智能诊断区和系统配置区分组，用户能够按照业务流程寻找功能。可维护性方面，后端使用蓝图拆分路由，服务层负责数据采集、诊断模型、知识库和风机数据仓库，模块边界较清晰。

扩展性方面，系统当前虽然使用 SQLite 和模拟数据，但数据接口、诊断服务和前端页面之间保持松耦合。后续如果接入真实风机数据，只需要替换数据采集服务或风机数据仓库，不需要大规模修改前端页面。若引入真实机器学习模型，也可以在 run_diagnosis 流程中替换风险评分和分类逻辑。

从可用性角度看，系统页面采用卡片、表格、图表和报告相结合的方式展示信息。对

于运维人员来说，卡片适合快速查看状态，表格适合追踪记录，图表适合观察趋势，报告适合形成结论。多种展示方式共同提高了系统在不同工作场景下的可读性。

表 6.5 非功能测试结果

测试项	测试方法	结果分析
稳定性	连续刷新页面和多次诊断	接口响应正常，页面无明显卡顿
可用性	按业务流程切换模块	导航清晰，核心功能易定位
可维护性	检查后端路由和服务划分	蓝图和服务层结构较清晰
扩展性	分析真实数据接入改造范围	可通过替换采集服务扩展
可解释性	查看诊断依据和建议措施	结果包含特征和风险说明

6.10 存在问题与改进方向

本系统已经实现齿轮箱健康管理的主要业务链条，但仍存在一些不足。首先，当前诊断数据主要来自模拟生成和统计文件，不能完全代表真实风场复杂工况。真实风机运行会受到风速波动、环境温度、功率限制、并网状态和检修状态等多种因素影响，后续需要接入真实 SCADA、CMS 和油液检测数据进行验证。

其次，当前故障诊断主要采用规则库和风险评分方法，优点是可解释性强，缺点是对复杂非线性退化过程表达能力有限。后续可以基于真实故障样本训练机器学习模型，例如支持向量机、随机森林、XGBoost、LSTM 或 Transformer 模型，并与规则库形成混合诊断机制。

再次，当前 RUL 估计仍属于启发式计算，能够展示寿命预测流程，但精度需要真实退化数据支撑。后续可以建立正常工况模型和退化趋势模型，结合剩余寿命标签进行监督学习。同时，应引入模型评价指标，如 MAE、RMSE、准确率、召回率和 F1 值，对模型性能进行量化评价。

最后，系统安全性和部署能力还有提升空间。毕业设计阶段主要在本地环境运行，后续若部署到真实风场，应增加 HTTPS、日志审计、权限细分、数据备份和异常恢复机制，并考虑服务容器化部署。这样才能从演示型系统进一步发展为可工程应用的智能健康管理平台。

第 7 章 总结与展望

7.1 研究总结

本文以华锐 SL1500 型双馈式风机齿轮箱为研究对象，完成了一套智能健康管理系统的设计与实现。系统基于 Flask、SQLite 和前端可视化技术，融合模拟采集数据和风机数据文件，构建了风场总览、单机详情、实时监测、故障诊断、故障代码库、故障数据管理、健康报告、参数设置、用户管理、HMI 和 AI 运维问答等功能模块。

在算法流程方面，系统实现了振动特征提取、风险因子计算、故障分类、健康评分和 RUL 估计。系统能够识别齿轮齿面磨损、齿面点蚀/剥落、齿轮断齿、轴承磨损、轴承点蚀/剥落、高速轴承过温、齿轮箱油温过高、润滑油污染/油液劣化、轴系不对中、齿轮啮合异常、箱体或基础松动和冷却系统故障等常见异常，并给出诊断依据和维护建议。

在工程实现方面，系统将后端接口、数据库模型、数据仓库、诊断服务、知识库和前端界面整合在一起，形成较完整的毕业设计原型。测试结果表明，系统能够完成主要业务流程，具备一定的演示价值和工程参考意义。

7.2 存在不足

本文系统仍存在以下不足：第一，诊断模型以规则和模拟数据为主，缺少真实故障样本训练和验证；第二，M-IALO-SVR、VMD-CNN 等算法尚未形成完整训练代码和评价结果，更多体现为系统预留和方法说明；第三，健康评分和 RUL 估计采用经验规则，尚未经过长期运行数据校准；第四，系统权限审计、数据安全、工单闭环和生产部署能力仍有待加强。

7.3 后续展望

后续研究可从以下方向继续完善。首先，接入真实 SCADA、CMS、油液检测和检修工单系统，建立标准化数据字典和样本库。其次，基于真实数据训练 M-IALO-SVR 油温残差预测模型，并与 SVR、随机森林、XGBoost、LSTM 等方法进行对比。再次，引入更多模型评价指标，如 MAE、RMSE、R2、准确率、召回率、F1 值和混淆矩阵，全面评价模型效果。最后，完善告警确认、检修派单、复测验收、权限审计和报表归档功能，使系统逐步从毕业设计原型向真实风电场运维辅助平台发展。

参考文献

- [1] 杨俊峰.小样本下基于深度神经网络的行星齿轮箱故障诊断[D].成都:电子科技大学,2024.DOI:10.27005/d.cnki.gdzku.2024.005469.
- [2] 周昶清.基于统计模型与稀疏表示的齿轮箱故障检测与诊断方法研究[D].杭州:浙江大学,2024.DOI:10.27461/d.cnki.gzjdx.2024.000307.
- [3] 武甲.基于深度学习与降维技术的风电机组齿轮箱状态监测可视化和健康评估[D].呼和浩特:内蒙古工业大学,2024.DOI:10.27225/d.cnki.gnmgu.2024.000206.
- [4] 许晴.基于时频分析和机器学习的齿轮箱故障诊断[D].北京:华北电力大学(北京),2024.
- [5] 吴岚.基于先进信号分析方法的风电机组齿轮箱故障诊断[D].北京:华北电力大学(北京),2024.
- [6] 马健.基于振动信号的风电机组齿轮箱特征提取方法和故障诊断研究[D].上海:上海电机学院,2024.
- [7] 吴迪.基于自动特征学习的风机齿轮箱故障诊断研究[D].沈阳:沈阳工业大学,2024.DOI:10.27322/d.cnki.gsgyu.2024.000493.
- [8] 孔德强.风电齿轮箱温度预警及关键部件故障诊断方法[D].沈阳:沈阳工程学院,2024.DOI:10.27845/d.cnki.gsygc.2024.000003.
- [9] 李常见.风电机组齿轮箱故障识别及安全状态评估研究[D].阜新:辽宁工程技术大学,2024.
- [10] 郭建超.风电机组齿轮箱轴承微弱故障诊断方法研究及系统开发[D].上海:上海电机学院,2024.
- [11] 王广.风力发电机组齿轮箱故障检测和寿命预测方法的研究[D].兰州:兰州理工大学,2024.
- [12] 范亚飞.基于变分模态分解和随机配置网络的齿轮箱故障诊断研究[D].石家庄:石家庄铁道大学,2024.DOI:10.27334/d.cnki.gstdy.2024.000257.
- [13] 杨青松.基于改进 ShuffleNet 的齿轮箱故障诊断研究[D].石家庄:石家庄铁道大学,2024.DOI:10.27334/d.cnki.gstdy.2024.000304.
- [14] 王莘然.基于迁移学习的风电齿轮箱故障诊断方法研究[D].大连:大连海事大学,2024.
- [15] 王丹.基于深度迁移学习的齿轮箱故障诊断方法研究[D].济南:齐鲁工业大学,2024.DOI:10.27278/d.cnki.gsdqc.2024.000503.

致谢

本毕业设计从选题、需求分析、系统设计、代码实现到论文撰写，得到了指导教师的悉心指导和同学的帮助。在完成课题过程中，本人进一步学习了风电机组齿轮箱故障诊断、Python Web 开发、数据库设计、前端可视化、数据分析和智能运维等知识。感谢指导教师在研究方向、功能设计和论文结构方面提出的宝贵意见，感谢同学在系统测试和问题反馈中的支持。由于本人水平有限，论文和系统仍存在不足之处，恳请各位老师批评指正。

附录 A 系统主要接口

表 A.1 系统主要接口

接口	功能
/health	后端服务健康检查
/api/auth/login	用户登录
/api/auth/register	用户注册
/api/data/status	获取实时运行状态
/api/data/vibration	获取振动波形和特征
/api/data/fault-codes	获取齿轮箱故障代码库
/api/data/diagnosis	执行故障诊断
/api/data/records	获取故障记录
/api/data/records/export	导出故障记录 CSV
/api/data/trend	获取故障趋势与分布
/api/windfarm/overview	获取风场总览
/api/windfarm/turbine/<unit_id>	获取单机详情
/api/report/generate	生成健康报告
/api/qa/ask	运维辅助问答
/api/ops/workorders	工单模拟接口

附录 B 核心代码片段与接口说明

本附录列出系统实现中具有代表性的核心代码片段，主要用于说明后端应用初始化、诊断接口组织、前端页面切换和实时数据刷新等关键实现。正文第 5 章已经给出部分算法代码，本附录进一步补充系统工程层面的代码结构，使论文内容与实际项目代码保持一致。

代码清单 B.1 Flask 应用初始化与蓝图注册

```
def create_app():
    app = Flask(__name__, static_folder='../frontend', static_url_path='')
    CORS(app)
    project_root = Path(app.root_path).resolve().parent
    database_path = project_root / 'instance' / 'gearbox_system.db'
    app.config['SQLALCHEMY_DATABASE_URI'] = f"sqlite:///{"database_path.as_posix()}"
    app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

    from models.database import init_db
    init_db(app)

    @app.route('/')
    def index():
        return app.send_static_file('index.html')

    app.register_blueprint(data_bp, url_prefix='/api/data')
    app.register_blueprint(windfarm_bp, url_prefix='/api/windfarm')
    app.register_blueprint(report_bp, url_prefix='/api/report')
    app.register_blueprint(qa_bp, url_prefix='/api/qa')
    app.register_blueprint(settings_bp, url_prefix='/api/settings')
    return app
```

代码清单 B.1 展示了系统后端入口的组织方式。Flask 应用创建后首先配置静态前端目录、数据库路径和跨域访问，然后初始化数据库模型，并按业务域注册多个蓝图。该结构使认证、数据、风场、报告、问答和设置接口相互独立，便于后续维护。

代码清单 B.2 故障诊断接口组织方式

```
@data_bp.route('/diagnosis', methods=['POST'])
def diagnose():
    payload = request.get_json(silent=True) or {}
    unit_id = payload.get('unit_id', 'WTG-001')
    scenario = payload.get('scenario', 'normal')
    raw_signal = generate_vibration_data(scenario=scenario)
    status = build_status_from_scenario(unit_id, scenario)
    result = run_diagnosis(unit_id=unit_id, raw_signal=raw_signal, status=status)
    record = FaultRecord(
        unit_id=unit_id,
        fault_type=result['fault_type'],
        severity=result['severity'],
        probability=result['probability'],
        health_score=result['health_score'],
        rul_days=result['rul_days'],
        advice=result['advice'],
        status='pending'
    )
    db.session.add(record)
    db.session.commit()
    return jsonify(result)
```

代码清单 B.2 体现了故障诊断接口的基本流程。接口接收前端传入的机组编号和诊断场景，生成或读取振动数据与状态数据，调用 `run_diagnosis` 完成诊断，并将结果写入故障记录表。这样可以保证诊断结果既能在页面展示，也能在故障数据管理模块中追踪。

代码清单 B.3 前端模块切换与视图控制

```
function showView(targetId) {
  const navItems = document.querySelectorAll('.nav-item');
  const sections = document.querySelectorAll('.view-section');
  navItems.forEach(item => {
    item.classList.toggle('active', item.dataset.target === targetId);
  });
  sections.forEach(section => {
    section.style.display = section.id === targetId ? 'block' : 'none';
    section.classList.toggle('active', section.id === targetId);
  });
  updateModuleIndicator(targetId);
}

document.querySelectorAll('.nav-item').forEach(item => {
  item.addEventListener('click', event => {
    event.preventDefault();
    showView(item.dataset.target);
  });
});
```

代码清单 B.3 展示了前端单页应用的视图切换方式。系统通过导航项的 `data-target` 属性确定目标模块，然后隐藏其他 `section` 并显示当前模块。该方式实现简单、依赖少，适合本系统这种本地演示型健康管理平台。

代码清单 B.4 健康报告关键指标渲染

```
async function fetchReport() {
  const unitId = document.getElementById('report-unit-select').value;
  const days = document.getElementById('report-range-select').value;
  const response = await fetch(`/api/report/generate?unit_id=${unitId}&days=${days}`);
  const report = await response.json();
  document.getElementById('report-health-score').textContent = report.health_score;
  document.getElementById('report-oil-temp').textContent = report.oil_temp + ' °C';
  document.getElementById('report-vibration').textContent = report.vibration_rms + ' mm/s';
  document.getElementById('report-rul').textContent = report.rul_days + ' 天';
  document.getElementById('report-conclusion').textContent = report.conclusion;
  renderReportTrend(report.trend);
}
```

代码清单 B.4 说明健康报告页面如何通过接口获取数据并渲染关键指标。报告页面不是静态文本，而是由后端根据当前机组状态、历史趋势和诊断记录动态生成。该实现使报告内容能够随系统数据变化而更新。

综上，附录 B 中的代码片段覆盖了系统启动、接口注册、诊断接口、前端模块切换和报告渲染等关键环节，与正文中的系统需求、总体设计、详细实现和测试分析相互对应，能够证明本文系统并非停留在概念设计层面，而是具有可运行的工程实现。

附录 C 系统部署与运行说明

本系统采用本地 Flask 服务和静态前端页面运行，适合毕业设计演示、功能测试和答辩展示。系统启动前应确认 Python 虚拟环境、依赖包、数据库目录和前端文件均存在。项目根目录下的 backend 目录保存后端程序，frontend 目录保存页面文件，instance 目录保存 SQLite 数据库。

启动系统时，进入项目根目录并执行后端入口文件。系统默认监听 127.0.0.1:5000 端口，启动后可通过浏览器访问首页。默认管理员账号用于演示登录，登录后可以查看风场总览、故障诊断、故障数据、健康报告、AI 问答和参数设置等模块。

```
cd "D:\bishe\Huaren SL1500 Type Doubly-Fed Wind Turbine Gearbox Intelligent Health Management System1.1"
```

```
.\.venv\Scripts\python.exe backend\app.py
```

```
http://127.0.0.1:5000
```

默认账号: admin

默认密码: admin

系统健康检查接口为/health，用于确认后端服务和数据库路径是否正常。若返回 status 为 ok，说明 Flask 服务已启动并能够访问数据库目录。系统页面中使用的实时数据、诊断接口、报告接口和问答接口均通过/api 前缀访问。

```
GET /health
```

```
GET /api/data/status
```

```
POST /api/data/diagnosis
```

```
GET /api/windfarm/overview
```

```
GET /api/report/generate
```

```
POST /api/qa/ask
```

答辩演示时建议先展示风场总览页面，说明系统能够对多台机组进行健康状态监测；随后进入单机详情页面，说明油温、振动、油液和 RUL 等指标；再进入故障诊断页面执行一次典型故障诊断；最后展示健康报告、AI 问答和参数设置页面，说明系统已经形成从监测、诊断、解释到维护建议的闭环。

附录 D 接口返回字段示例

为了便于说明前后端数据交互方式，本附录给出系统主要接口的返回字段示例。接口数据采用 JSON 格式，前端根据字段名称渲染卡片、表格、图表和报告内容。统一的字段结构有助于降低页面逻辑复杂度，也方便后续替换真实数据源。

```
{
  "unit_id": "WTG-001",
  "timestamp": "2026-05-24 20:00:22",
  "oil_temp": 68.4,
  "vibration_rms": 3.64,
  "oil_quality": 7,
  "health_score": 90,
  "rul_days": 376,
  "status": "normal",
  "alarm_level": "正常"
}
```

实时状态接口主要用于风场总览、单机详情和健康报告页面。字段中的 `oil_temp` 表示齿轮箱油温，`vibration_rms` 表示振动有效值，`oil_quality` 表示油液 NAS 等级，`health_score` 和 `rul_days` 用于展示健康评分和剩余寿命。

```
{
  "fault_type": "轴承点蚀/剥落",
  "severity": "严重",
  "component": "高速轴承",
  "probability": 0.86,
  "features": {
    "rms": 4.91,
    "kurtosis": 6.72,
    "crest_factor": 4.33,
    "envelope_peak": 1.48
  },
  "advice": "复核包络谱并安排现场检查"
}
```

故障诊断接口主要服务于诊断页面和故障记录页面。系统不仅返回故障类型，还返回严重程度、部件位置、置信度、振动特征和维护建议，从而保证诊断结果具有可解释性。